VOL. 01

HTML

Structure, content and semantics for the web

Source	html.com
Generated	2026-10-10
Level	Beginner
Estimated study time	20 hours
Language	English
Volume	Vol. 01 - HTML

This volume is personal study material. The notes, examples, exercises and tests were written from scratch while following the source site's learning path. No content is affiliated with, endorsed by, or copied from the source site or the technology's owner.

Table of contents

How to use this volume 7

Chapter map 7

Chapter 1 - How the web works and what HTML is 7

1.1 Summary 7

1.2 Concepts 8

The request cycle 8

Markup, not programming 8

Elements, tags and attributes 8

The three layers of a web page 9

HTML history in one paragraph 9

1.3 Examples 9

1.4 Common mistakes and best practices 11

1.5 Practice exercises 11

Solutions 11

1.6 Mini test 12

Answer key and explanations 12

Chapter 2 - Anatomy of an HTML document 12

2.1 Summary 12

2.2 Concepts 13

The skeleton 13

The head, element by element 13

The body 13

Language 14

Comments 14

2.3 Examples 14

2.4 Common mistakes and best practices 15

2.5 Practice exercises 15

Solutions 16

2.6 Mini test 16

Answer key and explanations 16

Chapter 3 - Text content 17

3.1 Summary 17

3.2 Concepts . . . 17
Text elements and when to use each . . . 17
Whitespace collapsing . . . 18
Semantic vs presentational . . . 18
3.3 Examples . . . 18
3.4 Common mistakes and best practices . . . 19
3.5 Practice exercises . . . 19
Solutions . . . 20
3.6 Mini test . . . 20
Answer key and explanations . . . 20
Chapter 4 - Links and navigation . . . 21
4.1 Summary . . . 21
4.2 Concepts . . . 21
URL forms . . . 21
Resolution example . . . 21
Anatomy of a link . . . 22
Accessibility of links . . . 22
4.3 Examples . . . 22
4.4 Common mistakes and best practices . . . 23
4.5 Practice exercises . . . 24
Solutions . . . 24
4.6 Mini test . . . 24
Answer key and explanations . . . 24
Chapter 5 - Lists . . . 25
5.1 Summary . . . 25
5.2 Concepts . . . 25
The three containers . . . 25
Ordered list attributes . . . 26
Nesting rules . . . 26
5.3 Examples . . . 26
5.4 Common mistakes and best practices . . . 27
5.5 Practice exercises . . . 28
Solutions . . . 28
5.6 Mini test . . . 28
Answer key and explanations . . . 28

Chapter 6 - Images and media 29
6.1 Summary 29
6.2 Concepts 29
The image element 29
Writing good `alt` text 29
Formats and when to use which 30
Figures and captions 30
Video and audio 30
6.3 Examples 30
6.4 Common mistakes and best practices 31
6.5 Practice exercises 32
Solutions 32
6.6 Mini test 32
Answer key and explanations 32
Chapter 7 - Tables 33
7.1 Summary 33
7.2 Concepts 33
Minimal structure 33
Full, accessible structure 33
Parts and attributes 34
7.3 Examples 34
7.4 Common mistakes and best practices 36
7.5 Practice exercises 36
Solutions 36
7.6 Mini test 37
Answer key and explanations 37
Chapter 8 - HTML5 semantics and page structure 37
8.1 Summary 37
8.2 Concepts 38
Landmark map of a typical page 38
Choosing between ``, `` and `` 38
The document outline 38
When `` and `` are not the page's 39
8.3 Examples 39
8.4 Common mistakes and best practices 41

8.5 Practice exercises . . . 42
Solutions . . . 42
8.6 Mini test . . . 42
Answer key and explanations . . . 43
Chapter 9 - Forms and user input . . . 43
9.1 Summary . . . 43
9.2 Concepts . . . 43
The form element . . . 43
Input types . . . 44
Common input attributes . . . 44
Labels and grouping . . . 44
9.3 Examples . . . 45
9.4 Common mistakes and best practices . . . 46
9.5 Practice exercises . . . 46
Solutions . . . 47
9.6 Mini test . . . 47
Answer key and explanations . . . 47
Chapter 10 - Entities, escaping and character sets . . . 48
10.1 Summary . . . 48
10.2 Concepts . . . 48
The characters you must escape . . . 48
Useful named entities . . . 48
Text direction and language . . . 49
10.3 Examples . . . 49
10.4 Common mistakes and best practices . . . 49
10.5 Practice exercises . . . 50
Solutions . . . 50
10.6 Mini test . . . 50
Answer key and explanations . . . 51
Chapter 11 - Accessibility, SEO and best practices . . . 51
11.1 Summary . . . 51
11.2 Concepts . . . 51
The accessibility tree . . . 51
Contrast and colour . . . 51
The page test . . . 52

SEO fundamentals . . . 52
11.3 Examples . . . 52
11.4 Common mistakes and best practices . . . 53
11.5 Practice exercises . . . 54
Solutions . . . 54
11.6 Mini test . . . 54
Answer key and explanations . . . 54
Chapter 12 - Mini project: a three-page personal site . . . 55
12.1 Summary . . . 55
12.2 Concepts . . . 55
Project structure . . . 55
What to build . . . 55
12.3 Examples . . . 55
Checks to run on the project . . . 60
12.4 Common mistakes and best practices . . . 60
12.5 Practice exercises . . . 60
Solutions . . . 60
12.6 Mini test . . . 61
Answer key and explanations . . . 61
Cheat sheet . . . 61
13.1 Document skeleton . . . 61
13.2 Text elements . . . 62
13.3 Links, images, media . . . 62
13.4 Lists . . . 63
13.5 Tables . . . 63
13.6 Semantics and landmarks . . . 63
13.7 Forms . . . 63
13.8 Entities and escaping . . . 64
13.9 Accessibility and SEO quick list . . . 64
Final test . . . 64
14.1 Final test answers . . . 65
Sources . . . 66

How to use this volume

Prerequisites: a computer with a text editor (VS Code, Notepad++, or even Notepad) and a browser. No prior programming knowledge is assumed.

Estimated study time: about 20 hours of reading, typing every example, doing the exercises and passing the final exam.

How the chapters work: each chapter starts with a summary you can read in two minutes, then explains the concepts, shows runnable examples, lists the common mistakes, and ends with practice exercises (with solutions) and a mini test with an answer key.

A rule for this volume: never copy and paste the examples. Type them. HTML punishes typos silently, and typing every tag is how you learn them.

Chapter map

#	Chapter	You will learn
1	How the web works and what HTML is	pages, browsers, tags, documents
2	Anatomy of an HTML document	the minimum document, head and body
3	Text content	headings, paragraphs, inline formatting
4	Links and navigation	anchors, relative and absolute paths
5	Lists	ordered, unordered, definition lists
6	Images and media	`img`, `figure`, audio and video
7	Tables	tabular data done right
8	HTML5 semantics and page structure	landmark elements, outline
9	Forms and user input	controls, labels, validation
10	Entities, escaping and character sets	safe text in HTML
11	Accessibility, SEO and best practices	semantics, alt text, validation
12	Mini project	a three-page personal site
13	Cheat sheet	one-page reference
14	Final test	25 questions with answers

Chapter 1 - How the web works and what HTML is

1.1 Summary

- The web is a set of documents (pages) that link to each other.
- A browser requests a page from a server, reads the HTML, and renders it.
- HTML is a markup language: you describe the meaning of content, not its look.
- Content lives in elements, written with tags: `<p>text</p>`.
- Tags can carry attributes: `<img src="cat.jpg" alt="A cat">`.
- HTML handles structure; CSS handles appearance; JavaScript handles behaviour.
- Modern HTML is one living standard maintained by WHATWG, no version numbers.
- Valid HTML is the foundation of accessibility, SEO and maintainability.

1.2 Concepts

The request cycle

When you type an address, this happens:

```
you -> browser -> DNS lookup -> server -> HTTP request
     -> HTML response -> browser renders the page
```

The browser receives HTML text and builds the DOM (Document Object Model), a tree of elements. CSS is applied on top of that tree, and JavaScript can change it.

Markup, not programming

HTML has no variables, loops or conditions. Every construct is a named element that describes what a piece of content is: a heading, a paragraph, a list, a table, a form control.

Bad question	Good question
How do I make this blue?	Is this a heading, or a paragraph?
How do I centre it?	Is this the main content, or navigation?
Which font size?	What is the importance of this text?

Appearance belongs in CSS (Vol. 02). Behaviour belongs in JavaScript (Vol. 03).

Elements, tags and attributes

An element has three parts:

HTML

```html
<a href="https://example.com" title="Example site">Example</a>
```

Part	Value here	Meaning
Start tag	`<a href="..." title="...">`	opens the element, with attributes
Content	`Example`	the text or children
End tag	`</a>`	closes the element

Some elements cannot have content and have no end tag (void elements), such as `<img>`, `<br>`, `<hr>`, `<input>`, `<meta>`, `<link>`.

Attributes come in four flavours:

Kind	Example	Notes
Boolean	`<input disabled>`	presence means true
Name/value	`lang="en"`	always quoted
URL	`href="/about"`	may be relative or absolute
Global	`id`, `class`, `title`, `hidden`	allowed on every element

The three layers of a web page

output

```
+-----------------------------------+
| HTML    structure and meaning     |
+-----------------------------------+
| CSS     presentation              |
+-----------------------------------+
| JS      behaviour                 |
+-----------------------------------+
```

If a layer is removed, the page should still work:

- Remove CSS: the page is ugly but complete.
- Remove JS: the page still delivers its content.

That is why HTML comes first in this library.

HTML history in one paragraph

Tim Berners-Lee created HTML around 1989 at CERN. It went through HTML 2, 3, 4 and XHTML, then HTML5 (2014) added the semantic elements (`<article>`, `<section>`, `<nav>`, `<header>`, `<footer>`), media elements (`<audio>`, `<video>`), form improvements, canvas and more. Since then the WHATWG maintains HTML as a living standard: there is no "HTML6", only the current standard.

1.3 Examples

Example 1: the smallest possible HTML page.

HTML

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>My first page</title>
</head>
<body>
  <p>Hello, world.</p>
</body>
</html>
```

Save it as `index.html` and open it in a browser. You should see the sentence "Hello, world." and the page title in the browser tab.

Expected output: a page containing one paragraph and the tab title `My first page`.

Example 2: seeing the DOM the browser built.

HTML

```html
<!-- Save as dom.html and open it -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>DOM demo</title>
</head>
<body>
  <h1>A heading</h1>
  <p>A paragraph with <strong>bold</strong> and <em>italic</em> text.</p>
</body>
</html>
```

Press `F12`, open the Elements panel, and you will see the same tags in a tree:

output

```
html
  head
    meta[charset=UTF-8]
    title "DOM demo"
  body
    h1 "A heading"
    p
      #text "A paragraph with "
      strong "bold"
```

Example 3: what happens with unclosed tags.

HTML

```html
<!-- BAD EXAMPLE on purpose: these paragraphs are never closed -->
<!DOCTYPE html>
<html lang="en">
<head><meta charset="UTF-8"><title>Broken</title></head>
<body>
  <p>This paragraph is never closed.
  <p>The browser closes it for me anyway.
  <p>Optional end tags are a legacy trap, not a licence.
</body>
</html>
```

Browsers recover from this, but the result is unpredictable across tools, screen readers and parsers. Always close what you open.

1.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Using `<b>`/`<i>` for emphasis	conveys looks, not meaning	use `<strong>`/`<em>`
Writing pages without `<meta charset>`	accented characters break	always set UTF-8
Treating HTML as visual design	unmaintainable, inaccessible	use CSS for appearance
Leaving tags unclosed	breaks parsing and SEO	close every element
Using `<table>` for page layout	inaccessible, fragile	use CSS layout
Choosing tags by their default font size	breaks the document outline	choose by meaning

- Do validate your pages: the W3C validator at `validator.w3.org` is free and explains every error.
- Do keep the three layers separate: one HTML file, one CSS file, one JS file.

1.5 Practice exercises

1. Write a complete minimal HTML page whose title is `Recipe` and whose body contains the text "Tomato soup".
2. Name the three parts of an element and mark them on `<a href="/next" class="link">Next</a>`.
3. List five void elements.
4. A colleague says HTML is "outdated because it cannot do loops". Explain in two sentences why this is a category error.
5. Given a page with `<p>one<p>two`, describe what a browser does and why you should not rely on it.

Solutions

1. `html <!DOCTYPE html> <html lang="en"> <head> <meta charset="UTF-8"> <title>Recipe</title> </head> <body> Tomato soup </body> </html>`
2. Start tag `<a href="/next" class="link">`, content `Next`, end tag `</a>`.
3. Any five of: `area`, `base`, `br`, `col`, `embed`, `hr`, `img`, `input`, `link`, `meta`, `source`, `track`, `wbr`.
4. HTML is a markup language, not a programming language: it describes the structure and meaning of content. Logic belongs in JavaScript (or the server language); generating repeated markup is exactly what a template or a script is for. Asking HTML for loops is asking a dictionary for multiplication.
5. The browser implicitly closes the first `<p>` when it sees the second one, so the two sentences become two sibling paragraphs. Because the behaviour is parser error recovery defined by legacy rules, different consumers (some screen readers, XML parsers, static

analysers) may treat it differently. Always write the end tags.

1.6 Mini test

1. Multiple choice: which layer of a web page describes meaning? - a) CSS - b) JavaScript - c) HTML - d) the DOM
2. True or false: HTML5 introduced version numbers that you must declare with a `version` attribute.
3. Multiple choice: which of these is a void element? - a) `<p>` - b) `<img>` - c) `<div>` - d) `<a>`
4. True or false: `<strong>` and `<b>` produce identical visual results, so they are interchangeable.
5. Short code question: fix this element so the attribute value is quoted correctly: `<a href=/about class=link>About</a>`.
6. Multiple choice: what does `<!DOCTYPE html>` declare? - a) the character set - b) the document type and that this is modern HTML - c) the language of the page - d) the CSS file to use
7. Ordering: put in order: browser parses HTML, server sends HTML, browser requests the page, browser renders the page.
8. Short answer: name two things that break on a page that omits `<meta charset="UTF-8">`.

Answer key and explanations

1. c - HTML describes structure and meaning; CSS presents, JS behaves.
2. False - HTML is now a living standard maintained by WHATWG with no version numbers; `<!DOCTYPE html>` is enough to trigger standards mode.
3. b - `img` is a void element: no content, no end tag.
4. False - both are bold by default, but `<strong>` marks importance while `<b>` only draws attention, which screen readers and search engines read differently.
5. `<a href="/about" class="link">About</a>` - attribute values must be quoted.
6. b - it tells the browser to use standards mode with the current HTML standard.
7. browser requests the page, server sends HTML, browser parses HTML, browser renders the page.
8. Any two of: accented letters and symbols display as mojibake, form data is submitted with the wrong encoding, the validator reports errors, the page may be interpreted as the wrong encoding by search engines.

Chapter 2 - Anatomy of an HTML document

2.1 Summary

- Every HTML document has the same skeleton: doctype, `html`, `head`, `body`.

- `<head>` holds metadata: title, charset, viewport, description, links to CSS.
- `<body>` holds everything the user sees.
- `<html lang="...">` tells assistive technology and translators the language.
- The viewport meta tag is what makes a page work on phones.
- One page, one `<title>`, one `<h1>`: the outline starts correctly.
- The doctype must be the very first thing in the file.
- Comments `<!-- -->` are ignored by the browser and useful for you.

2.2 Concepts

The skeleton

HTML

```html
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Page title</title>
    <meta name="description" content="One or two sentences.">
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <!-- visible content goes here -->
  </body>
</html>
```

The head, element by element

Element	Purpose	Required?
`<meta charset>`	encoding, must be within the first 1024 bytes	yes
`<meta name="viewport">`	mobile scaling	yes, for responsive pages
`<title>`	browser tab, bookmarks, search results	yes
`<meta name="description">`	the snippet under a search result	recommended
`<link rel="stylesheet">`	attaches CSS	when you use CSS
`<link rel="icon">`	favicon	recommended
`<meta name="author">`	authorship	optional

The body

Everything visible goes in the body: text, images, links, forms, tables. There is exactly one `<body>` per document.

Language

HTML

```html
<html lang="en">
```

Effects: screen readers pick the right voice, browsers choose the right hyphenation and quotes, and translation tools know the source language. Use the full code (`en`, `es`, `pt-BR`, `zh-Hans`), not just `en`.

Comments

HTML

```html
<!-- Remember to add the GDPR notice before publishing -->
```

Comments are invisible to users but visible to anyone who views the source, so never put secrets or internal data in them.

2.3 Examples

Example 1: a complete, realistic head.

HTML

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Ana Ruiz - Front-end developer</title>
  <meta name="description" content="Portfolio of Ana Ruiz, front-end developer
    specialising in accessible, fast websites.">
  <link rel="stylesheet" href="/assets/style.css">
  <link rel="icon" href="/favicon.svg" type="image/svg+xml">
</head>
<body>
  <h1>Ana Ruiz</h1>
  <p>Front-end developer.</p>
</body>
</html>
```

Expected result: the tab shows "Ana Ruiz - Front-end developer", the page renders a heading and a paragraph, and the CSS file is attached.

Example 2: what the title looks like in search results.

output

```
Ana Ruiz - Front-end developer
https://anaruiz.example
Portfolio of Ana Ruiz, front-end developer specialising in accessible,
fast websites.
```

This is why the title and description are written for humans, not for keywords.

Example 3: a heading structure for a two-level page.

HTML

```
<body>
  <h1>Ana Ruiz</h1>
  <h2>About</h2>
  <h2>Projects</h2>
  <h3>Lighthouse dashboard</h3>
  <h3>Recipe finder</h3>
  <h2>Contact</h2>
</body>
```

Rendered as a table of contents, the browser and screen readers see:

output

```
Ana Ruiz
  About
  Projects
    Lighthouse dashboard
    Recipe finder
  Contact
```

Never skip levels (h1 -> h3) and never use headings to make text big.

2.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Missing `lang`	wrong screen-reader voice	set `lang` on `<html>`
Forgetting the charset	broken characters	first meta tag
Duplicate titles	users cannot tell tabs apart	unique descriptive title per page
Using `<title>` stuffed with keywords	spammy, ignored	write it for a human
Putting visible content in `<head>`	invalid, unpredictable	only metadata in head
Reusing the same `h1` on every page	flat outline	one `h1` per page, matching the page topic
Comments with secrets	public in view-source	keep secrets in the server

- Keep the head order: charset, viewport, title, description, styles.
- Titles: front-load the meaningful words, keep them under about 60 characters.

2.5 Practice exercises

1. Write the head for a page titled `Cafe Aurora - Specialty coffee in Valencia` with a description and a stylesheet `main.css`.
2. Why must `<meta charset>` appear early in the document?
3. Turn this fake head into a valid one: `<head><title>Hi</title> <meta charset="utf-8"></head>` plus a viewport tag.

4. Explain why `<h1>` should be unique per page.
5. A page has `lang="es"` but the content is English. What breaks?

Solutions

1. `html <head> <meta charset="UTF-8"> <meta name="viewport" content="width=device-width, initial-scale=1"> <title>Cafe Aurora - Specialty coffee in Valencia</title> <meta name="description" content="Cafe Aurora roasts specialty coffee in Valencia. Beans, subscriptions and brewing guides."> <link rel="stylesheet" href="main.css"> </head>`
2. The browser must decide the encoding before it has parsed most of the document. The specification asks for the charset declaration inside the first 1024 bytes so that text is decoded correctly from the first character.
3. The tag itself is valid, but convention and safety put the charset first: `html <head> <meta charset="UTF-8"> <meta name="viewport" content="width=device-width, initial-scale=1"> <title>Hi</title> </head>`
4. Screen reader users navigate by headings; the first heading is the page identity. If three pages all start with the same `h1`, they cannot tell where they are, and the document outline loses its entry point.
5. The screen reader pronounces the English text with a Spanish voice (wrong phonemes and missing words), hyphenation is wrong, and translation tools mislabel the page.

2.6 Mini test

1. Multiple choice: which meta tag makes a page scale correctly on phones? - a) `<meta name="description">` - b) `<meta name="viewport" content="width=device-width, initial-scale=1">` - c) `<meta charset="UTF-8">` - d) `<link rel="stylesheet">`
2. True or false: you can have several `<title>` elements if the page has several sections.
3. Multiple choice: where does the CSS link belong? - a) in `<body>` - b) in `<head>` - c) after `</html>` - d) before the doctype
4. Short code question: write the tag that declares the page language as British English.
5. Multiple choice: what does a comment look like? - a) `// note` - b) `/* note */` - c) `<!-- note -->` - d) `-- note --`
6. True or false: `<h1>` may be used to make a big bold line anywhere.
7. Short answer: which two things does the `<title>` element feed?

Answer key and explanations

1. b - the viewport meta tag authorises the browser to lay the page out at the device width.
2. False - exactly one title per document.
3. b - in the head, so the CSS is applied before the first paint.
4. `<html lang="en-GB">`
5. c - `<!-- ... -->`.
6. False - headings express the document outline; styling belongs to CSS.

7. The browser tab or window title, and the title used by bookmarks, history and search engine results.

Chapter 3 - Text content

3.1 Summary

- Headings `<h1>` to `<h6>` create the document outline.
- Paragraphs are `<p>`; do not fake spacing with empty paragraphs.
- `<strong>` and `<em>` add meaning; `<b>` and `<i>` only change appearance.
- `<br>` breaks a line inside a paragraph; `<hr>` separates thematic sections.
- `<pre>` preserves spaces and line breaks, for code and ASCII art.
- `<blockquote>`, `<q>` and `<cite>` mark quoted material.
- `<code>`, `<kbd>`, `<samp>` and `<var>` mark up code and computer output.
- `<abbr>`, `<time>`, `<mark>`, `<sub>`, `<sup>`, `<del>` and `<ins>` cover the remaining common cases.
- Whitespace collapses: multiple spaces become one, newlines become a space.
- Text must be written for humans and for machines that read the outline.

3.2 Concepts

Text elements and when to use each

Element	Meaning	Visual default
`<h1>..<h6>`	heading of rank n	decreasing font size
`<p>`	paragraph	block with margins
`<strong>`	strong importance	bold
`<em>`	stress emphasis	italic
`<b>`	attention, no extra importance	bold
`<i>`	alternate voice, technical term	italic
`<mark>`	highlighted for reference	yellow background
`<small>`	side comment, fine print	smaller text
`<sub>` / `<sup>`	subscript / superscript	lowered / raised
`<del>` / `<ins>`	removed / added text	struck through / underlined
`<code>`	fragment of code	monospace
`<kbd>`	keyboard input	monospace
`<samp>`	program output	monospace

Element	Meaning	Visual default
`<var>`	variable name	italic or monospace
`<abbr title="...">`	abbreviation	dotted underline
`<time datetime="...">`	machine-readable date	plain text
`<blockquote>`	long quoted block	indented block
`<q>`	inline quotation	quotes
`<cite>`	title of a work	italic

Whitespace collapsing

HTML

```
<p>These     words
are    separated by many spaces.</p>
```

Renders as: "These words are separated by many spaces." The browser collapses runs of whitespace into a single space. To keep formatting, use `<pre>`.

Semantic vs presentational

HTML

```
<p>Do not press the red button. <strong>Seriously, do not.</strong></p>
<p>Press <kbd>Ctrl</kbd> + <kbd>R</kbd> to reload the page.</p>
<p>The function returned <samp>Error 404</samp>.</p>
```

Every element above adds information a machine can use, not just styling.

3.3 Examples

Example 1: an article opening with mixed inline elements.

HTML

```
<h2>Brewing the perfect espresso</h2>
<p>Espresso is made by forcing hot water through finely ground coffee at about
<strong>9 bars of pressure</strong>. The result should be <em>dense</em>, with a
layer of crema on top.</p>
<p>Water temperature matters: <mark>92 to 96 degrees Celsius</mark> is the usual
range. The chemical formula for water is H<sub>2</sub>O, and E = mc<sup>2</sup>
is unrelated but memorable.</p>
<p>As the baristas say, <q>the grinder is the machine</q>.</p>
```

Expected output: a level-2 heading, then paragraphs where "9 bars of pressure" is bold, "dense" is italic, the temperature range is highlighted, H2O uses a subscript 2 and E = mc2 uses a superscript.

Example 2: preserving layout with `<pre>`.

HTML

```
<pre>
  function greet(name) {
      return "Hello, " + name;
  }
</pre>
```

output

```
  function greet(name) {
      return "Hello, " + name;
  }
```

Everything between the tags keeps its spaces and line breaks exactly.

Example 3: semantic markup of a citation block.

HTML

```
<blockquote cite="https://example.com/spec">
  <p>A document is a tree of elements.</p>
</blockquote>
<p>Source: <cite>The HTML standard</cite>, section 3.</p>
```

Example 4: machine-readable dates.

HTML

```
<p>Published on <time datetime="2026-10-10">10 October 2026</time>.</p>
<p>The course starts at <time datetime="2026-10-10T09:30+02:00">9:30 in the
morning</time>.</p>
```

Humans read the visible text; search engines and calendars read the `datetime` attribute.

3.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
`<p>text  text</p>` for spacing	fake spacing, fragile	use CSS margins
Empty paragraphs for gaps	noise for screen readers	CSS
Headings chosen by size	broken outline	choose by rank
`<b>` for warnings	no semantics	`<strong>` plus CSS
`<pre>` for layout	unresponsive	CSS
Using entities instead of real characters	harder to maintain	type the character, set UTF-8

- Prefer real characters over entities for common symbols; entities make the source harder to read.
- Prefer `<time datetime>` whenever you publish a date.

3.5 Practice exercises

1. Mark up: "The <html> element is the root. Press Ctrl+S to save. The file is 3 rows and 4 columns." Use at least four different inline semantic elements.
2. Write a `<blockquote>` for the quote "Simplicity is a prerequisite for reliability" with a `cite` attribute pointing to a source URL.
3. Why does `<p>a</p><p> </p><p>b</p>` produce an unwanted empty line?
4. Render the output of a command-line program semantically.
5. Add a machine-readable date to "Updated on 14 March 2026".

Solutions

1. `html <p>The <code><html></code> element is the root. Press <kbd>Ctrl</kbd> + <kbd>S</kbd> to save. The table has <strong>3 rows</strong> and <mark>4 columns</mark>.</p>`
2. `html <blockquote cite="https://example.com/quote"> <p>Simplicity is a prerequisite for reliability.</p> </blockquote>`
3. The middle paragraph contains only a space, but it is still a real element with default top and bottom margins, so the browser draws a visible gap. Vertical spacing belongs to CSS.
4. `html <p>The program printed <samp>Done in 4.2s</samp>.</p>`
5. `<time datetime="2026-03-14">Updated on 14 March 2026</time>`

3.6 Mini test

1. Multiple choice: which element marks strong importance? - a) `<i>` - b) `<b>` - c) `<strong>` - d) `<mark>`
2. True or false: multiple spaces in HTML source are all rendered.
3. Multiple choice: which element preserves whitespace and line breaks? - a) `<code>` - b) `<pre>` - c) `<p>` - d) `<span>`
4. Short code question: write an element for a variable named `userId`.
5. Multiple choice: what does `<sub>` do? - a) superscript - b) subscript - c) strikethrough - d) underline
6. True or false: `<blockquote>` is for short inline quotations.
7. Short answer: which two elements carry human-readable text plus a machine-readable attribute?
8. Multiple choice: which is the best way to add vertical space between two paragraphs? - a) an empty paragraph - b) several `<br>` elements - c) a CSS margin - d) a horizontal rule

Answer key and explanations

1. c - `strong` adds importance; `b` is purely presentational.
2. False - runs of whitespace collapse to a single space.
3. b - `pre` is preformatted text.
4. `<var>userId</var>`
5. b - subscript, as in H2O.

6. False - `blockquote` is for long quoted blocks; `<q>` is inline.
7. `<abbr title="...">` and `<time datetime="...">` (also `<blockquote cite="...">`).
8. c - spacing is presentation, so it belongs in CSS.

Chapter 4 - Links and navigation

4.1 Summary

- Links are created with `<a href="...">`.
- `href` can be absolute, relative, root-relative, or a fragment.
- Relative URLs are resolved against the current page's URL.
- A fragment (`#section`) jumps to the element with that `id`.
- `mailto:` and `tel:` open the email app or the dialler.
- `target="_blank"` opens a new tab; add `rel="noopener"` for safety.
- `download` suggests saving the file instead of navigating.
- Link text must describe the destination: never "click here".
- An `<a>` without `href` is not a link.

4.2 Concepts

URL forms

Form	Example	Resolves to
Absolute	`https://example.com/x`	the given page
Root-relative	`/about`	`https://your-site.com/about`
Document-relative	`about.html`	same folder as the current page
Parent-relative	`../index.html`	one folder up
Fragment	`#contact`	element with `id="contact"` in the same page
Protocol	`mailto:a@b.c`, `tel:+34600111222`	other applications

Resolution example

Current page: `https://example.com/blog/2026/post.html`

href	Result
`other.html`	`https://example.com/blog/2026/other.html`
`../index.html`	`https://example.com/blog/index.html`

href	Result
`/contact`	`https://example.com/contact`
`#top`	`https://example.com/blog/2026/post.html#top`

Anatomy of a link

HTML

```html
<a href="/pricing" title="See our plans" rel="nofollow">Pricing</a>
```

Attribute	Meaning
`href`	destination
`title`	advisory tooltip (not reliable on touch)
`target`	`_blank`, `_self`, `_parent`, `_top` or a frame name
`rel`	relationship: `noopener`, `noreferrer`, `nofollow`, `alternate`, `licence`
`download`	download instead of navigate
`hreflang`	language of the destination
`type`	MIME type of the destination

Accessibility of links

- The link text is read out of context by screen reader users, so "read more" repeated ten times is useless.
- If the visible text is not enough, use `aria-label` or a visually hidden `<span>`, and remember that `title` is not a substitute.

4.3 Examples

Example 1: a small navigation with mixed URL forms.

HTML

```html
<nav>
  <a href="/">Home</a>
  <a href="/about.html">About</a>
  <a href="contact.html">Contact</a>
  <a href="#privacy">Privacy on this page</a>
  <a href="mailto:hello@example.com">Email us</a>
  <a href="tel:+34600111222">Call us</a>
</nav>

<section id="privacy">
  <h2>Privacy</h2>
</section>
```

Expected behaviour: the first two links use absolute site paths, `contact.html` is resolved relative to the current page, `#privacy` jumps to the section, and the last two open external

applications.

Example 2: a links page.

HTML

```html
<p>
  Read the <a href="https://html.spec.whatwg.org/"
  target="_blank" rel="noopener">HTML standard</a> or jump to
  <a href="chapters/forms.html">the forms chapter</a>.
</p>
```

`target="_blank"` plus `rel="noopener"` opens a new tab safely: the new page cannot access `window.opener`.

Example 3: a table of contents with fragments.

HTML

```html
<h1>Contents</h1>
<ul>
  <li><a href="#install">Installation</a></li>
  <li><a href="#usage">Usage</a></li>
</ul>

<h2 id="install">Installation</h2>
<p>Run the installer.</p>
<h2 id="usage">Usage</h2>
<p>Open the app.</p>
```

Example 4: a downloadable file.

HTML

```html
<a href="/files/report-2026.pdf" download="annual-report-2026.pdf">
  Download the annual report (PDF, 1.2 MB)
</a>
```

The visible text tells the user what they get and how big it is.

4.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
"Click here" links	meaningless out of context	describe the destination
`target="_blank"` without `rel`	security and UX	always add `rel="noopener"`
Absolute URLs for internal links	breaks in dev and staging	relative or root-relative
Linking a whole paragraph	hard to hit on touch	link the meaningful words
Duplicate `id` values	fragment jumps to the wrong place	keep ids unique
`href="#"` buttons	fake links	use `<button>`
Underlining non-link text	users try to click it	reserve underline for links

- Internal links should never require the protocol and domain name.

- Give the destination a `lang` hint when it is in another language, with `hreflang`.

4.5 Practice exercises

1. Write a link to `https://example.com/docs` with the text "the documentation" that opens in a new tab safely.
2. Resolve: current page `https://site.com/a/b/page.html`, href `../c.html`.
3. Why is `<a>Click here</a>` a bad link?
4. Write a link that starts an email with a subject line.
5. Make the third item of a list link to the element with `id="legal"`.

Solutions

1. `html <a href="https://example.com/docs" target="_blank" rel="noopener">the documentation</a>`
2. `https://site.com/a/c.html` - `..` goes up one directory, from `/a/b/` to `/a/`, then `c.html`.
3. Screen reader users often list the links of a page; a list full of "click here" tells them nothing about the destinations. It is also poor for search engines, which use the anchor text to understand the target page.
4. `html <a href="mailto:hello@example.com?subject=Question%20about%20the%20course"> Email us</a>` (Spaces must be percent-encoded as `%20`.)
5. `html <li><a href="#legal">Legal notice</a></li>`

4.6 Mini test

1. Multiple choice: which attribute makes an anchor a link? - a) `name` - b) `href` - c) `target` - d) `rel`
2. True or false: `<a name="top"></a>` is still the way to create a link target.
3. Multiple choice: current page `/blog/post.html`, href `../index.html` resolves to: - a) `/index.html` - b) `/blog/index.html` - c) `/post.html` - d) `/blog/index.html#post`
4. Short code question: write a link to `mailto:sales@example.com` with the body "Please send information".
5. Multiple choice: which `rel` value should accompany `target="_blank"`? - a) `nofollow` - b) `noopener` - c) `alternate` - d) `icon`
6. True or false: link text should describe where the link goes.
7. Short answer: what does the fragment `#contact` need in the page to work?

Answer key and explanations

1. b - without `href` the anchor is only a placeholder.
2. False - `name` on `<a>` is obsolete; use `id` on any element.
3. a - `/blog/` + `../` = `/`, then `index.html`.
4. `html <a href="mailto:sales@example.com?body=Please%20send%20information">Email sales</a>`

5. b - `noopener` prevents the opened page from controlling the original tab through `window.opener`.
6. True - because the text is read out of context.
7. An element with `id="contact"` somewhere in the same page.

Chapter 5 - Lists

5.1 Summary

- Three list types: `<ul>` unordered, `<ol>` ordered, `<dl>` description.
- Every item is `<li>`; in a `<dl>` the pair is `<dt>` and `<dd>`.
- `<ol>` has `type`, `start` and `reversed` attributes.
- Lists nest: the nested list goes inside the `<li>` of its parent.
- `<menu>` and `<dir>` are obsolete.
- Semantics matter: a list of links is navigation, a recipe's steps are ordered.
- Screen readers announce "list, 5 items", so do not fake lists with dashes.
- List markers can be styled with CSS; the choice of type is structural.

5.2 Concepts

The three containers

HTML

```html
<ul>
  <li>a</li>
  <li>b</li>
</ul>
```

HTML

```html
<ol>
  <li>first</li>
  <li>second</li>
</ol>
```

HTML

```html
<dl>
  <dt>HTML</dt>
  <dd>markup language</dd>
</dl>
```

Type	Use for	Default marker
`<ul>`	items where order does not matter	bullets
`<ol>`	steps, rankings, instructions	numbers

Type	Use for	Default marker
`<dl>`	term/definition pairs	none

Ordered list attributes

Attribute	Effect
`type="1"`	numbers (default)
`type="a"` / `type="A"`	lower / upper letters
`type="i"` / `type="I"`	roman numerals
`start="5"`	begin counting at 5
`reversed`	count downwards

Nesting rules

The nested list is a child of an `<li>`, never a sibling of `<ul>` or `<ol>`:

HTML

```
<ul>
  <li>Fruit
    <ul>
      <li>Apple</li>
      <li>Pear</li>
    </ul>
  </li>
  <li>Vegetables</li>
</ul>
```

Only `<li>` is allowed as a direct child of `ul` and `ol`. Only `<dt>` and `<dd>` are allowed in `dl`.

5.3 Examples

Example 1: a shopping list.

HTML

```
<h2>Shopping list</h2>
<ul>
  <li>Coffee beans</li>
  <li>Oat milk</li>
  <li>Bread</li>
</ul>
```

Expected output: three bullet points.

Example 2: instructions with a custom start and nested items.

HTML

```html
<h2>How to deploy</h2>
<ol type="1" start="3">
  <li>Build the assets
    <ol>
      <li>Compile the CSS</li>
      <li>Bundle the JavaScript</li>
    </ol>
  </li>
  <li>Upload to the server</li>
</ol>
```

Rendered: step 3 "Build the assets" with two nested sub-steps, then step 4 "Upload to the server".

Example 3: a glossary.

HTML

```html
<dl>
  <dt>Element</dt>
  <dd>A part of the document marked up with tags.</dd>
  <dt>Attribute</dt>
  <dd>Extra information placed in a start tag.</dd>
  <dt>Void element</dt>
  <dd>An element that cannot have content, such as <code>img</code>.</dd>
</dl>
```

Example 4: roman numeral steps.

HTML

```html
<ol type="I" start="4">
  <li>Prepare the dough</li>
  <li>Rest for 30 minutes</li>
  <li>Bake</li>
</ol>
```

Expected output: IV, V, VI.

5.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
`<div>` + dashes instead of `<ul>`	no list semantics for screen readers	use real lists
Text directly inside `<ul>`	invalid	wrap in `<li>`
`<ol>` for unrelated items	wrong meaning	use `<ul>`
Faking numbers in text	loses structure	use `<ol>`
`<dl>` without `<dt>`/`<dd>`	invalid	pair the elements
Deeply nested lists	hard to navigate	restructure with headings

- Use `<ol>` for anything the user must follow in order.
- Style markers with `list-style-type` instead of changing the element.

5.5 Practice exercises

1. Mark up three unrelated features with an unordered list.
2. Mark up a three-step installation with an ordered list starting at 1.
3. Nest a sub-list of two items inside the second item of an ordered list.
4. Write a `<dl>` with two terms: `HTML` and `CSS`.
5. Why is `<ul><li>a</li>text<li>b</li></ul>` invalid?

Solutions

1. `<ul> <li>Works offline</li> <li>No account needed</li> <li>Dark mode</li> </ul>`
2. `<ol> <li>Download the archive</li> <li>Extract it</li> <li>Run <code>npm install</code></li> </ol>`
3. `<ol> <li>First step</li> <li>Second step <ul> <li>Sub-item one</li> <li>Sub-item two</li> </ul> </li> </ol>`
4. `<dl> <dt>HTML</dt> <dd>The markup language that describes document structure.</dd> <dt>CSS</dt> <dd>The language that describes presentation.</dd> </dl>`
5. `ul` may only contain `li` elements. The stray text node between the two items is a content model violation, which the validator reports as an error.

5.6 Mini test

1. Multiple choice: which element is required inside `<ul>`? - a) `<p>` - b) `<li>` - c) `<div>` - d) `<ol>`
2. True or false: a `<dl>` contains `<dt>` and `<dd>` elements.
3. Multiple choice: which attribute makes an ordered list count down? - a) `start="-3"` - b) `reversed` - c) `type="I"` - d) `dir="rtl"`
4. Short code question: start an ordered list at 10 with roman numerals.
5. Multiple choice: where does a nested list belong? - a) inside the parent `<ul>`, after the last `<li>` - b) inside an `<li>` of the parent list - c) inside a `<div>` - d) anywhere
6. True or false: the default marker of `<ol>` is a bullet.
7. Short answer: why prefer real lists over lines starting with a dash?

Answer key and explanations

1. b - the content model of `ul` is zero or more `li` elements.
2. True - and each group may have several `dd` per `dt`.
3. b - `reversed` makes the numbering descend.
4. `<ol type="I" start="10">` - renders X, IX, VIII...
5. b - the nested list is part of the parent item's content.
6. False - the default is decimal numbers.
7. Real lists are announced with the item count, can be navigated by assistive technology, and can be styled from CSS.

Chapter 6 - Images and media

6.1 Summary

- Images use `<img src="..." alt="...">`, a void element.
- `alt` is mandatory: describe the image, or use `alt=""` for decoration.
- `width` and `height` attributes prevent layout shift.
- `srcset` and `sizes` let the browser choose the right resolution.
- `<picture>` swaps formats or art direction with `<source>`.
- `<figure>` + `<figcaption>` attach a caption.
- `<video>` and `<audio>` play media; always add `<track>` for captions.
- Formats: JPEG for photos, PNG for transparency, SVG for icons, WebP/AVIF for modern size wins.
- `<map>` + `<area>` creates image maps; `<canvas>` is for scripted graphics.
- Never hotlink other people's bandwidth, and respect licences.

6.2 Concepts

The image element

HTML

```html
<img src="cat.jpg" alt="A grey cat sleeping on a keyboard" width="800"
     height="600" loading="lazy">
```

Attribute	Meaning
`src`	URL of the image
`alt`	text alternative (required)
`width` / `height`	intrinsic size, reserves space
`loading="lazy"`	defer off-screen images
`decoding="async"`	decode off the main thread
`srcset` / `sizes`	responsive candidates
`referrerpolicy`	how much referrer to send

Writing good `alt` text

Image kind	alt
Informative photo	"A grey cat sleeping on a keyboard"
Logo in the header	"Acme Corp home" (it is a link)

Image kind	alt
Icon that repeats a label	`alt=""` (the label carries the meaning)
Decorative divider	`alt=""`
Chart	"Sales doubled from 2024 to 2026"
Linked image	describe the destination, not the picture

Formats and when to use which

Format	Best for	Transparency	Animation
JPEG	photographs	no	no
PNG	screenshots, logos needing alpha	yes	no
SVG	icons, logos, illustrations	yes	yes (CSS/SMIL)
WebP	modern replacement for JPEG/PNG	yes	yes
AVIF	best compression today	yes	yes

Figures and captions

HTML

```html
<figure>
  <img src="chart.svg" alt="Sales doubled between 2024 and 2026">
  <figcaption>Figure 1. Annual revenue.</figcaption>
</figure>
```

Video and audio

HTML

```html
<video controls width="640" poster="poster.jpg">
  <source src="clip.mp4" type="video/mp4">
  <source src="clip.webm" type="video/webm">
  <track kind="captions" src="captions.vtt" srclang="en" label="English">
  Your browser does not support video.
</video>
```

6.3 Examples

Example 1: a responsive image with `srcset`.

HTML

```html
<img
  src="photo-800.jpg"
  srcset="photo-400.jpg 400w, photo-800.jpg 800w, photo-1600.jpg 1600w"
  sizes="(max-width: 600px) 100vw, 600px"
  alt="The harbour at dawn with three fishing boats"
  width="800" height="533" loading="lazy">
```

On a phone the browser downloads `photo-400.jpg`; on a desktop, `photo-800.jpg` or `photo-1600.jpg` depending on the density.

Example 2: format fallback with `<picture>`.

HTML

```
<picture>
  <source srcset="logo.avif" type="image/avif">
  <source srcset="logo.webp" type="image/webp">
  <img src="logo.png" alt="Acme Corp" width="200" height="80">
</picture>
```

Browsers that understand AVIF load `logo.avif`; the others fall back to PNG.

Example 3: an audio player.

HTML

```
<audio controls src="interview.mp3">
  <a href="interview.mp3">Download the interview</a>
</audio>
```

Example 4: an inline SVG icon.

HTML

```
<svg width="16" height="16" viewBox="0 0 16 16" aria-hidden="true"
     focusable="false">
  <circle cx="8" cy="8" r="7" fill="none" stroke="currentColor"/>
</svg>
```

Expected output: a small circle outline; `aria-hidden` hides it from screen readers because the adjacent text already labels the control.

6.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Missing `alt`	accessibility failure, SEO loss	always write `alt` (empty if decorative)
`alt="image"` or `alt="photo"`	useless to the user	describe the content
No width/height	layout shift (CLS)	always set both
Huge JPEGs for thumbnails	slow pages	resize and compress
Text in images	unreadable, unselectable	real HTML text
Hotlinking external images	unreliable, rude	host your own
Autoplaying video with sound	hostile UX	never autoplay with sound

- Add `loading="lazy"` to below-the-fold images, but never to the hero image.
- Provide captions with `<track>` for every video.

6.5 Practice exercises

1. Write an `<img>` for `team.jpg` with appropriate alt text and dimensions.
2. Write a `<figure>` containing an image and a caption "Figure 2: the tower".
3. Which `alt` is correct for a decorative divider?
4. Write a `<picture>` that prefers WebP and falls back to JPEG.
5. Why must `alt` not start with "image of"?

Solutions

1. `html <img src="team.jpg" alt="Five team members around a table in the workshop" width="1200" height="800" loading="lazy">`
2. `html <figure> <img src="tower.jpg" alt="A stone tower seen from the south" width="600" height="900"> <figcaption>Figure 2: the tower</figcaption> </figure>`
3. `alt=""` - an empty alternative tells assistive technology to ignore it.
4. `html <picture> <source srcset="photo.webp" type="image/webp"> <img src="photo.jpg" alt="A harbour at dawn"> </picture>`
5. Because screen readers already announce that it is an image. The value of `alt` is the information the image carries: "a harbour at dawn", not "image of a harbour at dawn".

6.6 Mini test

1. Multiple choice: which attribute is required on `<img>`? - a) `width` - b) `alt` - c) `loading` - d) `srcset`
2. True or false: `alt=""` is the right choice for a decorative image.
3. Multiple choice: which format is best for an icon? - a) JPEG - b) SVG - c) BMP - d) GIF
4. Short code question: lazy-load an image called `map.jpg`.
5. Multiple choice: what do `<figure>` and `<figcaption>` do together? - a) add a border - b) group an image with its caption - c) create a slideshow - d) preload the image
6. True or false: `srcset` lets the browser choose between image candidates.
7. Short answer: which two elements make a video caption available to assistive technology?

Answer key and explanations

1. b - `alt` is required (it may be empty for decorative images).
2. True - an empty alternative removes the image from the accessibility tree without breaking the page.
3. b - SVG scales without loss and can be styled.
4. `<img src="map.jpg" alt="..." loading="lazy">`
5. b - a figure is a self-contained unit with an optional caption.
6. True - the browser picks based on viewport and pixel density.
7. `<source>` for the media files and `<track kind="captions">` for the text track.

Chapter 7 - Tables

7.1 Summary

- Tables are for tabular data, never for page layout.
- Structure: `<table>`, `<tr>`, `<th>`, `<td>`.
- Group rows with `<thead>`, `<tbody>` and `<tfoot>`.
- `<caption>` gives the table an accessible name.
- `<th scope="col|row">` tells assistive technology what a header applies to.
- `colspan` and `rowspan` merge cells.
- Border and spacing styling belongs to CSS, not attributes.
- Simple tables are more accessible than deeply merged ones.
- Print the table without CSS: if it still makes sense, the structure is right.

7.2 Concepts

Minimal structure

HTML

```html
<table>
  <tr>
    <th>City</th>
    <th>Population</th>
  </tr>
  <tr>
    <td>Valencia</td>
    <td>800,000</td>
  </tr>
</table>
```

Full, accessible structure

HTML

```html
<table>
  <caption>Population in 2026</caption>
  <thead>
    <tr>
      <th scope="col">City</th>
      <th scope="col">Population</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th scope="row">Valencia</th>
      <td>800,000</td>
    </tr>
    <tr>
      <th scope="row">Seville</th>
      <td>690,000</td>
    </tr>
  </tbody>
  <tfoot>
    <tr>
      <th scope="row">Total</th>
      <td>1,490,000</td>
    </tr>
  </tfoot>
</table>
```

Parts and attributes

Element	Role
`<caption>`	title of the table
`<colgroup>` / `<col>`	column grouping and shared styling
`<thead>`	header rows
`<tbody>`	body rows (one or several)
`<tfoot>`	summary rows
`scope="col"`	header for the cells below it
`scope="row"`	header for the cells to its right
`colspan` / `rowspan`	cell spanning

7.3 Examples

Example 1: a table with a spanning header.

HTML

```html
<table>
  <caption>Quarterly results</caption>
  <thead>
    <tr>
      <th scope="col">Region</th>
      <th scope="col" colspan="2">Revenue (EUR)</th>
    </tr>
    <tr>
      <th scope="col"></th>
      <th scope="col">2025</th>
      <th scope="col">2026</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th scope="row">North</th>
      <td>120,000</td>
      <td>180,000</td>
    </tr>
    <tr>
      <th scope="row">South</th>
      <td>95,000</td>
      <td>140,000</td>
    </tr>
  </tbody>
</table>
```

Expected output: a table where "Revenue (EUR)" spans the two year columns, and the first row uses the empty header as a spacer.

Example 2: rowspan for multi-row categories.

HTML

```html
<table>
  <caption>Sprint backlog</caption>
  <thead>
    <tr>
      <th scope="col">Owner</th>
      <th scope="col">Task</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th scope="row" rowspan="2">Ana</th>
      <td>Write the importer</td>
    </tr>
    <tr>
      <td>Test the importer</td>
    </tr>
    <tr>
      <th scope="row">Luis</th>
      <td>Design the certificate</td>
    </tr>
  </tbody>
</table>
```

Example 3: a table with a footer total and `<col>` styling.

HTML

```html
<table>
  <caption>Invoice</caption>
  <colgroup>
    <col style="width:60%">
    <col style="width:15%">
    <col style="width:25%">
  </colgroup>
  <thead>
    <tr><th scope="col">Item</th><th scope="col">Qty</th>
        <th scope="col">Price</th></tr>
  </thead>
  <tbody>
    <tr><td>Coffee</td><td>2</td><td>8.00 EUR</td></tr>
    <tr><td>Tea</td><td>1</td><td>3.50 EUR</td></tr>
  </tbody>
  <tfoot>
    <tr><td>Total</td><td></td><td>11.50 EUR</td></tr>
  </tfoot>
</table>
```

7.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Tables for layout	unreadable and fragile	CSS layout
No `<th>`	no programmatic headers	use `<th scope>`
`<table border="1">`	presentational, obsolete	CSS borders
Deep `colspan`/`rowspan` nests	screen readers get lost	split into two tables
Missing `<caption>`	table has no name	always add one
Layout cells for spacing	noise	CSS

- Keep one header per column; add `scope` rather than relying on position.
- For wide tables on phones, add `overflow-x: auto` in CSS rather than restructuring the data.

7.5 Practice exercises

1. Build a two-column table of three cities with a caption.
2. Which element names a table for screen readers?
3. Write the header cell for a column of prices with correct scope.
4. Why is `<table border="1" cellpadding="4">` discouraged?
5. Convert a "price list laid out with a table that also holds the site menu" into the correct approach.

Solutions

1. `html <table> <caption>Three cities</caption> <thead> <tr><th scope="col">City</th><th scope="col">Country</th></tr> </thead> <tbody> <tr><th`

```
scope="row">Valencia</th><td>Spain</td></tr> <tr><th
scope="row">Lyon</th><td>France</td></tr> <tr><th
scope="row">Turin</th><td>Italy</td></tr> </tbody> </table>
```

2. `<caption>`
3. `<th scope="col">Price</th>`
4. Those are presentational attributes from HTML 4. They mix appearance with structure, were deprecated in HTML5, and are replaced by CSS, which also allows one style change to affect every table.
5. Keep the price data in a real table with `<caption>`, `<th scope>` and `<tbody>`, and move the site menu to a `<nav>` with a `<ul>`. The table then only contains tabular data, and layout and menu styling live in CSS.

7.6 Mini test

1. Multiple choice: which element is a data cell? - a) `<th>` - b) `<td>` - c) `<tr>` - d) `<caption>`
2. True or false: tables may be used to lay out a page header.
3. Multiple choice: which attribute declares that a header applies to the column below it? - a) `headers` - b) `scope="col"` - c) `colspan` - d) `role`
4. Short code question: add a caption "Budget" to a table.
5. Multiple choice: which group contains the header rows? - a) `<tbody>` - b) `<thead>` - c) `<tfoot>` - d) `<colgroup>`
6. True or false: `<colgroup>` shares styling across columns.
7. Short answer: what should you do with a table that has five nested `colspan`s?

Answer key and explanations

1. b - `td` is the data cell, `th` is the header cell.
2. False - tables are for tabular data; layout belongs to CSS.
3. b - `scope="col"` (and `scope="row"` for row headers).
4. `<caption>Budget</caption>` as the first child of `<table>`.
5. b - `thead`.
6. True - via `<col>` children of `<colgroup>`.
7. Split it into several simpler tables, or move to a definition list; complex merges defeat the accessibility tree.

Chapter 8 - HTML5 semantics and page structure

8.1 Summary

- Semantic elements describe the role of a region, not its position.

- Landmarks: `<header>`, `<nav>`, `<main>`, `<footer>`, `<aside>`, `<section>`, `<article>`, `<hgroup>`, `<search>`.
- `<main>` is unique per page and skips repetitive blocks.
- `<section>` needs an accessible name (a heading); `<div>` needs none.
- `<article>` is a self-contained, reusable piece of content.
- `<aside>` is tangentially related content: sidebars, pull quotes, ads.
- One `<h1>`, then `<h2>` per section, then `<h3>`: the outline follows the nesting.
- Old layout (`<table>`, `<font>`, framesets) is obsolete.
- `<header>` inside `<article>` is that article's header, not the page's.

8.2 Concepts

Landmark map of a typical page

output

```
header            (banner, usually site-wide)
  nav             (navigation)
main              (main content)
  article
    header       (article title and metadata)
    section      (chapter, needs a heading)
    footer       (article footer)
  aside            (related links)
footer             (contentinfo)
```

Choosing between `<div>`, `<section>` and `<article>`

Question	If yes use
Is it a generic box for styling or scripting only?	`<div>`
Is it a thematic group that deserves to appear in the outline?	`<section>`
Could this content be syndicated on its own and still make sense?	`<article>`
Is it the dominant content of the page?	`<main>`
Is it navigation?	`<nav>`
Is it tangentially related?	`<aside>`

The document outline

Headings create an outline that assistive technology exposes. With sectioning elements, the outline is unambiguous:

HTML

```html
<main>
  <h1>Kitchen</h1>
  <section>
    <h2>Bread</h2>
    <article>
      <h3>Sourdough</h3>
    </article>
  </section>
</main>
```

output

```
1. Kitchen
   1.1 Bread
      1.1.1 Sourdough
```

When `<header>` and `<footer>` are not the page's

HTML

```html
<article>
  <header>
    <h2>Post title</h2>
    <p>By Ana, 10 October 2026</p>
  </header>
  <p>Body...</p>
  <footer>
    <p>Tags: html, semantics</p>
  </footer>
</article>
```

Here `header` and `footer` are scoped to the article.

8.3 Examples

Example 1: a complete semantic page.

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Ana Ruiz - Front-end developer</title>
</head>
<body>
  <header>
    <p class="logo"><a href="/">Ana Ruiz</a></p>
    <nav aria-label="Main">
      <ul>
        <li><a href="/">Home</a></li>
        <li><a href="/about.html">About</a></li>
        <li><a href="/contact.html">Contact</a></li>
      </ul>
    </nav>
  </header>

  <main>
    <h1>I build fast, accessible websites</h1>

    <article>
      <header>
        <h2>Lighthouse dashboard</h2>
        <p>Published <time datetime="2026-03-14">14 March 2026</time></p>
      </header>
      <p>A dashboard that tracks Core Web Vitals for ten client sites.</p>
      <section>
        <h3>Problem</h3>
        <p>The agency had no single place to see performance regressions.</p>
      </section>
      <section>
        <h3>Solution</h3>
        <p>A nightly job that stores the metrics and a page that draws them.</p>
      </section>
      <footer>
        <p>Tags: performance, monitoring</p>
      </footer>
    </article>
  </main>

  <aside aria-label="Related projects">
    <h2>Related</h2>
    <ul>
      <li><a href="/projects/recipe-finder.html">Recipe finder</a></li>
    </ul>
  </aside>

  <footer>
    <p>(c) 2026 Ana Ruiz</p>
  </footer>
</body>
</html>
```

Expected structure, as reported by an accessibility tree viewer:

output

```
banner
  navigation (Main)
main
  heading level 1
  article
    heading level 2
    heading level 3 (Problem)
    heading level 3 (Solution)
complementary (Related projects)
contentinfo
```

Example 2: semantic vs non-semantic.

HTML

```html
<!-- Bad: no meaning, only boxes -->
<div class="header">
  <div class="nav"><div class="item">Home</div></div>
</div>
<div class="content">
  <div class="title">Hello</div>
</div>

<!-- Good: the roles are explicit -->
<header>
  <nav aria-label="Main"><a href="/">Home</a></nav>
</header>
<main>
  <h1>Hello</h1>
</main>
```

Example 3: `<section>` with a heading versus a bare `<div>`.

HTML

```html
<section id="pricing">
  <h2>Pricing</h2>
  <p>Two plans, no hidden fees.</p>
</section>

<div class="card-grid">
  <p>Cards are laid out with grid.</p>
</div>
```

The section appears in the outline because it has a heading; the grid does not, because it is purely a styling hook.

8.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
`<div>` for everything	no outline, no landmarks	use semantic elements
`<section>` without a heading	an unnamed landmark	add a heading or use `<div>`
Multiple `<main>`	invalid, confuses assistive tech	exactly one

Mistake	Why it hurts	Best practice
`<nav>` for every list of links	too many navigation landmarks	one per navigation region
Using `<header>` twice at page level	one banner per page	`<header>` only for scoped headers
`<article>` for a paragraph	over-semantic	`<section>` or `<p>`
Style hooks as the only reason for `<section>`	outline noise	`<div>`

- Verify with a browser extension that shows landmarks, or with a screen reader.
- Landmark elements plus correct heading levels are the cheapest accessibility win available.

8.5 Practice exercises

1. Mark up a page skeleton: header with navigation, main with an article, aside, footer.
2. Which element is the dominant content landmark?
3. Why does `<section id="pricing"><div>...</div></section>` (no heading) fail best practice?
4. Choose between `<article>` and `<section>` for a full blog post.
5. Add an accessible name to a `<nav>`.

Solutions

1. `html <body> <header><nav aria-label="Main"><a href="/">Home</a></nav></header> <main> <article> <h2>Post</h2> <p>Body</p> </article> </main> <aside aria-label="Related"> <h2>Related</h2> </aside> <footer><p>(c) 2026</p></footer> </body>`
2. `<main>`
3. A `<section>` is a region of the document outline, and a region without a heading gives assistive technology an unnamed landmark. Either add an `<h2>` or change it to a `<div>`, which is the neutral container.
4. `<article>` - it is self-contained and could be syndicated.
5. `<nav aria-label="Main">` (or `<nav aria-labelledby="...">` pointing at a visible heading).

8.6 Mini test

1. Multiple choice: how many `<main>` elements may a page have? - a) one per section - b) exactly one - c) two, one per column - d) as many as needed
2. True or false: `<article>` is for self-contained, syndicatable content.
3. Multiple choice: which element marks tangentially related content? - a) `<aside>` - b) `<aside role="main">` - c) `<section>` - d) `<footer>`
4. Short code question: give a `<nav>` the accessible name "Footer".

5. Multiple choice: which is the styling-only container? - a) `<section>` - b) `<article>` - c) `<div>` - d) `<aside>`
6. True or false: a `<section>` without a heading is valid HTML.
7. Short answer: what does `<header>` mean when it is inside `<article>`?

Answer key and explanations

1. b - exactly one, otherwise the landmark map is ambiguous.
2. True - a post, a comment, a product card.
3. a - `aside`, the complementary landmark.
4. `<nav aria-label="Footer">`
5. c - `div` has no semantics.
6. True - it validates, but the accessibility tree gains an unnamed region, so best practice is to add a heading or use `div`.
7. The header of that article (title, byline, date), not the page banner.

Chapter 9 - Forms and user input

9.1 Summary

- A form is `<form>` plus controls; everything else is labelling.
- Every control needs a `<label>` tied to it with `for` + `id`.
- Inputs have `type`: text, email, password, number, date, checkbox, radio.
- `<select>` is a dropdown, `<textarea>` is multi-line text.
- `<fieldset>` + `<legend>` group related controls, especially radios.
- HTML validation: `required`, `min`, `max`, `pattern`, `minlength`.
- Submit with `<button type="submit">` or `<input type="submit">`.
- Validation is a UX aid; the server must always re-validate.
- Explain errors with text, tied with `aria-describedby`.

9.2 Concepts

The form element

HTML

```html
<form action="/subscribe" method="post">
  <label for="email">Email</label>
  <input type="email" id="email" name="email" required>
  <button type="submit">Subscribe</button>
</form>
```

Attribute	Meaning
`action`	URL that processes the submission
`method`	`get` (in the URL) or `post` (in the body)
`enctype`	`application/x-www-form-urlencoded` (default), `multipart/form-data` (files)
`novalidate`	skip browser validation

Input types

Type	Use
`text`	single line
`email`, `tel`, `url`	validated text
`password`	masked
`number`, `range`	numeric
`date`, `time`, `month`	dates
`checkbox`	0..n choices
`radio`	1 of n choices
`file`	upload
`search`	search field
`submit`, `reset`, `button`	buttons

Common input attributes

Attribute	Effect
`required`	must not be empty
`minlength` / `maxlength`	length limits
`min` / `max` / `step`	numeric and date bounds
`pattern`	regular expression
`placeholder`	hint (not a label)
`readonly`	visible, not editable
`disabled`	not editable, not submitted
`autocomplete`	browser autofill token
`inputmode`	virtual keyboard hint on mobile

Labels and grouping

HTML

```html
<fieldset>
  <legend>Shipping method</legend>
  <label for="std"><input type="radio" id="std" name="ship" value="std"
    checked> Standard</label>
  <label for="exp"><input type="radio" id="exp" name="ship" value="exp">
    Express</label>
</fieldset>
```

Wrapping the input inside the label makes the whole text clickable; the `for`/`id` pair is what ties them together for assistive technology.

9.3 Examples

Example 1: a complete contact form.

HTML

```html
<form action="/contact" method="post">
  <p>
    <label for="name">Name</label>
    <input type="text" id="name" name="name" autocomplete="name" required
           minlength="2">
  </p>
  <p>
    <label for="email">Email</label>
    <input type="email" id="email" name="email" autocomplete="email" required>
  </p>
  <p>
    <label for="topic">Topic</label>
    <select id="topic" name="topic">
      <option value="sales">Sales</option>
      <option value="support" selected>Support</option>
    </select>
  </p>
  <p>
    <label for="msg">Message</label>
    <textarea id="msg" name="msg" rows="5" required maxlength="500"></textarea>
  </p>
  <p>
    <input type="checkbox" id="copy" name="copy" value="yes">
    <label for="copy">Send me a copy</label>
  </p>
  <button type="submit">Send</button>
</form>
```

Submitting an empty form makes the browser block it and report that the name, the email and the message are required, and that the email must contain `@`.

Example 2: native validation messages.

HTML

```html
<form action="/login" method="post">
  <label for="pw">Password</label>
  <input type="password" id="pw" name="pw" minlength="8" required>
  <button type="submit">Log in</button>
</form>
```

Submitting `123` is refused with "Please lengthen this text to 8 characters or more" and nothing is sent to the server.

Example 3: a search form.

HTML

```html
<form action="/search" method="get" role="search">
  <label for="q">Search</label>
  <input type="search" id="q" name="q" placeholder="e.g. espresso">
  <button type="submit">Go</button>
</form>
```

The query lands in the URL as `/search?q=espresso`, so results are bookmarkable.

Example 4: grouping radios.

HTML

```html
<fieldset>
  <legend>Delivery</legend>
  <label for="d1"><input type="radio" id="d1" name="d" value="home" checked>
    Home</label>
  <label for="d2"><input type="radio" id="d2" name="d" value="pickup">
    Pickup</label>
</fieldset>
```

Both radios share `name="d"`, so choosing one deselects the other, and the legend names the group for screen readers.

9.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Placeholder instead of label	disappears while typing, no accessible name	always use a real `<label>`
Divs as labels	nothing ties them to the input	`<label for>`
Unique `name` per radio	all can be selected at once	same `name` for the group
Div with a click handler as button	no keyboard, no role	`<button type="submit">`
Trusting browser validation	easily bypassed	re-validate on the server
`autocomplete="off"` everywhere	breaks password managers	use the right tokens

- Use `fieldset`/`legend` for radio and checkbox groups.
- Keep the tab order equal to the visual order.

9.5 Practice exercises

1. Write a labelled email input that is required.
2. Build a two-radio group "Yes"/"No" inside a fieldset named "Newsletter".

3. What is wrong with `<input type="text" placeholder="Your name">` alone?
4. Write a search form using the `get` method.
5. Add a max length of 20 to a username input.

Solutions

1. `html <label for="email">Email</label> <input type="email" id="email" name="email" required>`
2. `html <fieldset> <legend>Newsletter</legend> <label for="ny"><input type="radio" id="ny" name="news" value="yes"> Yes</label> <label for="nn"><input type="radio" id="nn" name="news" value="no"> No</label> </fieldset>`
3. There is no `<label>`, so assistive technology has no accessible name for the field, and the placeholder vanishes as soon as the user types.
4. `html <form action="/search" method="get" role="search"> <label for="q">Search</label> <input type="search" id="q" name="q"> <button type="submit">Go</button> </form>`
5. `<input type="text" id="username" name="username" maxlength="20">`

9.6 Mini test

1. Multiple choice: which attribute links a label to an input? - a) `name` - b) `for` - c) `id` on the label - d) `control`
2. True or false: radios in one group must share a `name`.
3. Multiple choice: which element groups related inputs with a name? - a) `<legend>` - b) `<fieldset>` - c) `<optgroup>` - d) `<label>`
4. Short code question: a required text input for a full name.
5. Multiple choice: which `method` puts the data in the URL? - a) `post` - b) `get` - c) both - d) neither
6. True or false: `placeholder` replaces `label`.
7. Short answer: why must the server re-validate everything?

Answer key and explanations

1. b - `for` on the label plus the matching `id` on the control.
2. True - the shared name is what makes them alternatives.
3. b - `fieldset` groups, `legend` names the group.
4. `<input type="text" id="fullname" name="fullname" required>`
5. b - `get` appends the fields to the query string.
6. False - placeholder is a hint; label is the accessible name.
7. Client-side validation is a convenience that can be disabled, forged or bypassed (no JavaScript, `novalidate`, a crafted request). The server is the only place where data becomes trustworthy.

Chapter 10 - Entities, escaping and character sets

10.1 Summary

- `<meta charset="UTF-8">` makes most entities unnecessary.
- Entities start with `&` and end with `;`.
- The five core entities are `&` `<` `>` `"` `'`.
- You must escape `<`, `>` and `&` in text.
- Numeric references work too: `©` and `©`.
- Emoji are characters; no entities needed with UTF-8.
- `lang` and `dir` control language and text direction.
- Never use entities for formatting, such as ` ` for spacing.
- Wrongly escaped user content is an XSS vulnerability, not a cosmetic bug.

10.2 Concepts

The characters you must escape

Character	Entity	Why
`&`	`&`	starts an entity
`<`	`<`	starts a tag
`>`	`>`	ends a tag
`"`	`"`	in double-quoted attribute values
`'`	`'`	in single-quoted attribute values

Useful named entities

Entity	Character	Name
`©`	(c)	copyright
`®`	(R)	registered
`™`	TM	trademark
`€`	EUR	euro
` `		non-breaking space
`…`	...	ellipsis
`—`	--	em dash

Entity	Character	Name
`“ ”`	curly quotes	

Text direction and language

HTML

```
<p lang="ar" dir="rtl">Arabic text runs right to left.</p>
<p>Mixed: the word <span lang="ar" dir="rtl">arabic</span> inside English.</p>
```

`dir` accepts `ltr`, `rtl` and `auto`; `auto` guesses from the first strong character.

10.3 Examples

Example 1: showing code on a page.

HTML

```
<p>To print a paragraph in HTML, write <code>&amp;lt;p&amp;gt;text&amp;lt;/p&amp;gt;</code>.</p>
```

Renders as: "To print a paragraph in HTML, write `<p>text</p>`."

Example 2: a character table.

You type	You see
`<`	`<`
`© 2026`	(c) 2026
`5 km`	5 km (never breaks the line)

Example 3: the trap of double escaping.

HTML

```
<!-- Wrong: shows &amp; on the page -->
<p>Fish &amp;amp; chips</p>

<!-- Right: shows & -->
<p>Fish &amp; chips</p>
```

Example 4: escaping user content (why servers template-escape).

HTML

```
<!-- If a user typed <script>alert(1)</script> into a comment field and the
     server did not escape it, this would execute. -->
<p>The comment says: &lt;script&gt;alert(1)&lt;/script&gt;</p>
```

The browser renders the text `<script>alert(1)</script>` instead of executing it. Escaping on output is the first defence against XSS.

10.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
` ` for spacing	breaks wrapping and copy-paste	CSS margins
Entities for accented letters	unreadable source	type the character with UTF-8
Forgetting `&` in URLs	the query string breaks	escape ampersands in `href`
Entities in unquoted attributes	invalid HTML	always quote attributes
Printing raw user input	XSS	escape on output, always

10.5 Practice exercises

1. Write the escaped version of: `Use <b> for bold & ignore the rest`.
2. Which entity produces a copyright sign?
3. Escape this attribute value safely: `<a title="He said "hi"">x</a>`.
4. Why is `<meta charset="UTF-8">` still needed if entities exist?
5. Write a Hebrew paragraph with the right attributes.

Solutions

1. `Use <b> for bold & ignore the rest`
2. `©` or `©`.
3. `html <a title="He said "hi"">x</a>` (or use single quotes around the value and `"` inside)
4. Entities are unreliable for real text: authors forget them, and editors and translation tools mangle them. UTF-8 lets you type any character directly, keeps the source readable, and covers emoji and accented letters alike.
5. `<p lang="he" dir="rtl">...</p>`

10.6 Mini test

1. Multiple choice: which entity is a literal ampersand? - a) `∧` - b) `&` - c) `&` - d) `&#38`
2. True or false: with UTF-8 you should still type `é`.
3. Multiple choice: which characters must always be escaped in text? - a) `"` only - b) `<`, `>` and `&` - c) spaces - d) punctuation
4. Short code question: show `5 < 6` as text on a page.
5. Multiple choice: what is the safest default for server templates? - a) never escape - b) escape everything, mark trusted content explicitly - c) escape only `&` - d) escape only in attributes
6. True or false: ` ` is a good way to add horizontal space.
7. Short answer: which two attributes control direction and language?

Answer key and explanations

1. b - `&` is the entity for `&`.
2. False - with UTF-8 you type the character directly.
3. b - those are the markup-significant characters.
4. `5 < 6`
5. b - escape by default and deliberately opt out for trusted content.
6. False - it prevents line breaks, which breaks reflow and selection. Use CSS.
7. `dir` and `lang` (`dir="rtl"`, `lang="he"`).

Chapter 11 - Accessibility, SEO and best practices

11.1 Summary

- Accessible HTML is semantic HTML done properly.
- Landmarks and heading levels are the navigation of screen reader users.
- `alt` text, labels and captions are not optional.
- Keyboard focus must always be visible; never remove the focus ring without replacing it.
- Contrast: 4.5:1 for normal text (WCAG AA).
- SEO basics: one `h1`, descriptive title and description, semantic structure, fast pages, working links.
- Validate with the W3C validator; test with the keyboard and a screen reader.
- Accessibility is a continuous practice, not a final checklist.

11.2 Concepts

The accessibility tree

Browsers expose a parallel tree of landmarks, headings, names, roles and states:

output

```
page
  banner
    navigation "Main"
  main
    heading level 1 "I build fast, accessible websites"
    article
      heading level 2 "Lighthouse dashboard"
      link "Read more"
    textbox "Email" (required)
  contentinfo
```

Contrast and colour

Text size	Minimum ratio (AA)
Normal text	4.5:1
Large text (18.66px bold or 24px)	3:1
UI components and graphics	3:1

Never encode meaning in colour alone: add text, an icon or a pattern.

The page test

1. Can I reach every control with `Tab` and see where I am?
2. Does every image have meaningful (or empty) `alt`?
3. Does every form control have a visible label?
4. Are the headings in order?
5. Does the page still make sense with CSS disabled?

SEO fundamentals

Factor	What to do
Title	unique, descriptive, front-loaded
Meta description	one or two sentences written for humans
Heading structure	one `h1`, then `h2` per section
Semantic HTML	landmarks, lists, tables for data
Links	descriptive anchor text, no broken links
Performance	compressed images, minimal CSS and JS
Mobile	responsive viewport, readable text size

11.3 Examples

Example 1: a focus-visible style (never a bare `outline: none`).

CSS

```css
:focus-visible {
  outline: 2px solid #0969da;
  outline-offset: 2px;
}
```

HTML

```html
<a href="/next" class="btn">Next page</a>
```

Example 2: a skip link.

HTML

```html
<body>
  <a href="#main" class="skip-link">Skip to content</a>
  <header>...</header>
  <main id="main">
    <h1>Page title</h1>
  </main>
</body>
```

Keyboard users go straight to the content instead of tabbing through the whole navigation on every page.

Example 3: accessible names in practice.

HTML

```html
<!-- Bad: no name at all -->
<button><svg width="16" height="16" aria-hidden="true"><path d="..."/></svg></button>

<!-- Good: the icon is decorative, the label is real -->
<button type="submit">
  <svg width="16" height="16" aria-hidden="true" focusable="false">
    <path d="..."/>
  </svg>
  <span>Search</span>
</button>
```

Example 4: a table that is readable as text.

HTML

```html
<table>
  <caption>Course prices</caption>
  <thead>
    <tr><th scope="col">Course</th><th scope="col">Price</th></tr>
  </thead>
  <tbody>
    <tr><th scope="row">HTML</th><td>2 EUR</td></tr>
  </tbody>
</table>
```

Without CSS the reading order is still "Course: HTML, Price: 2 EUR".

11.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
`outline: none` on focus	keyboard users get lost	style `:focus-visible`
Colour-only error states	invisible to many users	icon plus text plus colour
Divs as buttons	no keyboard, no role	`<button>`
Missing `alt`	information loss	write `alt` (empty if decorative)
Auto-playing carousels	WCAG 2.2.2 violation	user control, pause button
Tiny text	unreadable	16px minimum body size
"Click here"	poor for screen readers and SEO	descriptive link text

11.5 Practice exercises

1. Add a skip link to a page whose main content has `id="content"`.
2. Give a label to an icon-only submit button.
3. Why is `outline: none` on `:focus` harmful?
4. Write the two metadata elements every page should have for SEO.
5. A form marks errors in red only. What is missing?

Solutions

1. `html <a href="#content" class="skip-link">Skip to content</a> ... <main id="content">`
2. `html <button type="submit">Search</button>` with the icon marked `aria-hidden="true"`.
3. The focus ring is the only visual cue telling keyboard users which element will be activated. Removing it without an obvious replacement makes the page unusable without a mouse.
4. `<title>` and `<meta name="description">` (plus the head basics: charset and viewport).
5. A text or icon indication. Colour is the only signal, so users who cannot distinguish it, or who use a screen reader, get no information. Add an explicit message next to the field and tie it with `aria-describedby`.

11.6 Mini test

1. Multiple choice: minimum contrast ratio for normal text at WCAG AA? - a) 2:1 - b) 3:1 - c) 4.5:1 - d) 7:1
2. True or false: `alt=""` is valid and useful for decorative images.
3. Multiple choice: which element should an interactive control be? - a) `<div>` - b) `<span>` - c) `<button>` or `<a>` - d) `<p>`
4. Short code question: write a skip link to `#main`.
5. Multiple choice: how many `h1` elements should a normal page have? - a) zero - b) one - c) one per section - d) one per paragraph
6. True or false: autoplaying a video with sound is acceptable if it is short.
7. Short answer: name two things a screen reader uses to navigate.

Answer key and explanations

1. c - 4.5:1 for normal text, 3:1 for large text.
2. True - it removes the image from the accessibility tree.
3. c - real interactive elements carry role, focus and keyboard behaviour.
4. `<a href="#main" class="skip-link">Skip to content</a>`
5. b - one, matching the page topic.
6. False - WCAG 2.2.2 requires a way to pause or stop moving or auto-playing content, and audio must never start without user action.
7. Any two of: landmarks, heading levels, link text, form labels, the DOM reading order.

Chapter 12 - Mini project: a three-page personal site

12.1 Summary

- A real page is the skeleton of Chapter 2 with every other chapter applied.
- Consistent header and footer, one CSS file, three HTML files.
- Navigation, semantic main content, a list, an image, a table and a form.
- Every image has `alt`; every control has a label; the outline is correct.
- Each page has a unique title and description.

12.2 Concepts

Project structure

output

```
site/
  index.html
  about.html
  contact.html
  assets/
    main.css
    portrait.jpg
```

What to build

File	Contents
`index.html`	header, nav, hero, list of services, image
`about.html`	article with sections, a table of skills, a blockquote
`contact.html`	form with text, email, select, textarea, checkbox

12.3 Examples

`index.html`:

HTML

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Ana Ruiz - Front-end developer</title>
  <meta name="description" content="Front-end developer building fast,
    accessible websites. Projects, notes and contact.">
  <link rel="stylesheet" href="assets/main.css">
</head>
<body>
  <header>
    <p><a href="/">Ana Ruiz</a></p>
    <nav aria-label="Main">
      <ul>
        <li><a href="/" aria-current="page">Home</a></li>
        <li><a href="about.html">About</a></li>
        <li><a href="contact.html">Contact</a></li>
      </ul>
    </nav>
  </header>

  <main>
    <h1>I build fast, accessible websites</h1>
    <p>Ten years of front-end work for agencies and small teams.</p>

    <h2>What I do</h2>
    <ul>
      <li>Accessible interfaces</li>
      <li>Front-end architecture</li>
      <li>Performance audits</li>
    </ul>

    <img src="assets/portrait.jpg" alt="Ana Ruiz in her studio" width="480"
         height="480">
  </main>

  <footer>
    <p>(c) 2026 Ana Ruiz</p>
  </footer>
</body>
</html>
```

`about.html`:

HTML

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>About - Ana Ruiz</title>
  <meta name="description" content="Background, skills and working method.">
  <link rel="stylesheet" href="assets/main.css">
</head>
<body>
  <header>
    <p><a href="/">Ana Ruiz</a></p>
    <nav aria-label="Main">
      <ul>
        <li><a href="/">Home</a></li>
        <li><a href="about.html" aria-current="page">About</a></li>
        <li><a href="contact.html">Contact</a></li>
      </ul>
    </nav>
  </header>

  <main>
    <article>
      <h1>About me</h1>

      <section>
        <h2>Background</h2>
        <p>I started as a designer and moved to the front end because I wanted
          to control the result.</p>
      </section>

      <section>
        <h2>Skills</h2>
        <table>
          <caption>Main skills</caption>
          <thead>
            <tr><th scope="col">Skill</th><th scope="col">Years</th></tr>
          </thead>
          <tbody>
            <tr><th scope="row">HTML and CSS</th><td>10</td></tr>
            <tr><th scope="row">JavaScript</th><td>8</td></tr>
            <tr><th scope="row">Accessibility</th><td>5</td></tr>
          </tbody>
        </table>
      </section>

      <section>
        <h2>Method</h2>
        <ol>
          <li>Understand the content</li>
          <li>Structure it semantically</li>
          <li>Style and test</li>
        </ol>
      </section>
    </article>
  </main>
```

HTML (continued)

```
57
58    <footer>
59      <p>(c) 2026 Ana Ruiz</p>
60    </footer>
61  </body>
62  </html>
```

contact.html:

HTML

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Contact - Ana Ruiz</title>
  <meta name="description" content="Get in touch about a project.">
  <link rel="stylesheet" href="assets/main.css">
</head>
<body>
  <header>
    <p><a href="/">Ana Ruiz</a></p>
    <nav aria-label="Main">
      <ul>
        <li><a href="/">Home</a></li>
        <li><a href="about.html">About</a></li>
        <li><a href="contact.html" aria-current="page">Contact</a></li>
      </ul>
    </nav>
  </header>

  <main>
    <h1>Contact</h1>
    <form action="/contact" method="post">
      <p>
        <label for="name">Name</label>
        <input type="text" id="name" name="name" autocomplete="name" required>
      </p>
      <p>
        <label for="email">Email</label>
        <input type="email" id="email" name="email" autocomplete="email"
               required>
      </p>
      <p>
        <label for="topic">Topic</label>
        <select id="topic" name="topic">
          <option value="project">A project</option>
          <option value="talk">A talk</option>
          <option value="other">Something else</option>
        </select>
      </p>
      <p>
        <label for="message">Message</label>
        <textarea id="message" name="message" rows="6" required
                  maxlength="1000"></textarea>
      </p>
      <p>
        <input type="checkbox" id="newsletter" name="newsletter" value="yes">
        <label for="newsletter">Send me the occasional newsletter</label>
      </p>
      <button type="submit">Send</button>
    </form>
  </main>

  <footer>
    <p>(c) 2026 Ana Ruiz</p>
```

HTML (continued)

```html
  </footer>
</body>
</html>
```

Checks to run on the project

1. Every file validates at `validator.w3.org`.
2. `Tab` reaches the skip link, the three nav links, then every control.
3. Each image has `alt`; each input has a `label`.
4. Titles differ on all three pages.
5. With CSS disabled the structure still reads as an outline.

12.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Copy-pasting the header into every page	drift between pages	duplicate the markup deliberately, or generate it
Same `<title>` on all three pages	tabs and history become useless	unique, descriptive titles
Linking the CSS with a different path per page	broken styles on one page	one relative path from the site root, same folder depth
Forgetting the `alt` on the new image	accessibility failure	describe the image, or use `alt=""`
Repeating the form markup without labels	unusable forms	label every control, group with `fieldset`
Marking the current page with a class only	invisible to assistive tech	`aria-current="page"`
Leaving the mini project without validation	errors accumulate	validate before publishing

- Keep the three files in one folder so the relative links work identically.
- After the project works, delete one page and re-create it from memory: that is the real test of whether the structure has sunk in.

12.5 Practice exercises

1. Add a `privacy.html` page and link it in the footer of all three pages.
2. Add `<time datetime="2026-10-10">` to the about page "last updated" line.
3. Add a `<figure>` with a caption to the index page.
4. Convert the services `<ul>` into an ordered list and say when that is right.
5. Add `loading="lazy"` where appropriate and say where it must not be used.

Solutions

1. `html <footer> <p>(c) 2026 Ana Ruiz</p> <p><a href="privacy.html">Privacy</a></p> </footer>`
2. `<p>Last updated <time datetime="2026-10-10">10 October 2026</time></p>`

3. `html <figure> <img src="assets/portrait.jpg" alt="Ana Ruiz in her studio" width="480" height="480"> <figcaption>In the studio, autumn 2026.</figcaption> </figure>`
4. It is right when the order matters (steps, ranking). For a list of services where nothing precedes anything else, keep `<ul>`.
5. Use `loading="lazy"` for images below the fold. Never for the hero image: lazy-loading it delays the largest visible element and hurts LCP.

12.6 Mini test

1. Multiple choice: what links a site's CSS? - a) `<script src="...">` - b) `<link rel="stylesheet" href="...">` - c) `<style href="...">` - d) `<meta name="css">`
2. True or false: all three pages may share the same `<title>`.
3. Multiple choice: which attribute marks the current page in a nav? - a) `aria-current="page"` - b) `class="active"` - c) `selected` - d) `href="#"`
4. Short code question: link to `assets/main.css` from a page at the root.
5. Multiple choice: which element holds the page's main content? - a) `<header>` - b) `<div class="content">` - c) `<main>` - d) `<section>`
6. True or false: an image above the fold should be lazy-loaded.
7. Short answer: name two checks you run before publishing.

Answer key and explanations

1. b - a `link` element in the head.
2. False - unique, descriptive titles per page.
3. a - `aria-current="page"` announces it to assistive technology.
4. `<link rel="stylesheet" href="assets/main.css">`
5. c - `<main>`.
6. False - hero images should load eagerly so they do not delay LCP.
7. Any two of: validate the HTML, check the keyboard path, check `alt` and labels, test with CSS disabled, check the titles, run Lighthouse.

Cheat sheet

13.1 Document skeleton

HTML

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Page title</title>
  <meta name="description" content="...">
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header><nav aria-label="Main">...</nav></header>
  <main>...</main>
  <footer>...</footer>
</body>
</html>
```

13.2 Text elements

Element	Use
`h1`..`h6`	headings, one `h1` per page
`p`	paragraph
`strong` / `em`	importance / emphasis
`b` / `i`	attention / alternate voice
`mark`	highlighted reference
`small`	side comment
`sub` / `sup`	sub / superscript
`del` / `ins`	removed / added
`pre`	preformatted text
`code` / `kbd` / `samp` / `var`	code / keys / output / variable
`abbr title`	abbreviation
`time datetime`	machine-readable date
`blockquote cite`	quoted block
`q`	inline quote
`cite`	work title
`br` / `hr`	line break / thematic break

13.3 Links, images, media

HTML

```html
<a href="url" target="_blank" rel="noopener">text</a>
<a href="mailto:a@b.c?subject=Hi">email</a>
<a href="#id">fragment</a>
<img src="x.jpg" alt="description" width="800" height="600" loading="lazy">
<picture>
  <source srcset="x.webp" type="image/webp">
  <img src="x.jpg" alt="description">
</picture>
<figure><img src="x.jpg" alt="..."><figcaption>Caption</figcaption></figure>
<video controls src="x.mp4"><track kind="captions" src="c.vtt"
  srclang="en" label="English"></video>
```

13.4 Lists

HTML

```html
<ul><li>item</li></ul>
<ol type="1" start="3"><li>item</li></ol>
<dl><dt>term</dt><dd>definition</dd></dl>
```

13.5 Tables

HTML

```html
<table>
  <caption>Title</caption>
  <thead><tr><th scope="col">A</th><th scope="col">B</th></tr></thead>
  <tbody><tr><th scope="row">r</th><td>v</td></tr></tbody>
  <tfoot><tr><td>totals</td></tr></tfoot>
</table>
```

13.6 Semantics and landmarks

Element	Meaning
`header`	banner (or scoped header)
`nav`	navigation, needs a name if repeated
`main`	dominant content, exactly one
`article`	self-contained content
`section`	thematic group, needs a heading
`aside`	complementary content
`footer`	contentinfo (or scoped footer)

13.7 Forms

HTML

```html
<form action="/x" method="post">
  <label for="a">Text field</label>
  <input type="text" id="a" name="a" required minlength="2">
  <label for="b">Dropdown</label>
  <select id="b" name="b"><option value="1">One</option></select>
  <label for="c">Multi-line</label>
  <textarea id="c" name="c" rows="4" maxlength="500"></textarea>
  <fieldset><legend>Group</legend>
    <input type="radio" id="r1" name="g" value="1"><label for="r1">One</label>
    <input type="checkbox" id="k1" name="k" value="1"><label for="k1">Yes</label>
  </fieldset>
  <button type="submit">Send</button>
</form>
```

13.8 Entities and escaping

`& < > " ' © ®   —`

- In text: escape `<`, `>`, `&`.
- In double-quoted attributes: also escape `"`.
- Always escape user data on output, on the server.

13.9 Accessibility and SEO quick list

- One `h1`, ordered headings, landmarks, skip link.
- `alt` on every image (empty when decorative).
- A real `<label>` for every control.
- Visible focus styles, contrast 4.5:1, no colour-only meaning.
- Unique `<title>` and a real `<meta name="description">`.
- Descriptive link text, no broken links.

Final test

1. Multiple choice: which declaration must be first in an HTML file? - a) `<html>` - b) `<!DOCTYPE html>` - c) `<head>` - d) `<meta charset>`
2. True or false: `<strong>` and `<b>` have the same semantics.
3. Multiple choice: an `<img>` without `alt` is... - a) fine if the image is small - b) invalid - c) only a problem for SEO - d) acceptable in footers
4. Short code question: write a link to `https://example.com` with the text "Example site" that opens safely in a new tab.
5. Multiple choice: which element is required as a child of `<ul>`? - a) `<p>` - b) `<li>` - c) `<div>` - d) `<ol>`
6. Multiple choice: which attribute makes a table header apply to the cells below it? - a) `headers` - b) `scope="col"` - c) `colspan` - d) `data-col`
7. True or false: tables are suitable for laying out a page header.

8. Short code question: start an ordered list at 5 with uppercase roman numerals.
9. Multiple choice: how many `<main>` elements may a page contain? - a) one per section - b) exactly one - c) two - d) unlimited
10. Multiple choice: which element gives a table an accessible name? - a) `<title>` - b) `<caption>` - c) `<legend>` - d) `aria-name`
11. True or false: `placeholder` can replace a `label`.
12. Short code question: write a required email input with a label.
13. Multiple choice: which entity is a literal ampersand? - a) `∧` - b) `&` - c) `&#38` - d) `&`
14. Multiple choice: minimum contrast ratio for normal text? - a) 2:1 - b) 3:1 - c) 4.5:1 - d) 10:1
15. True or false: `target="_blank"` requires `rel="noopener"` for safety.
16. Short code question: escape the text `5 < 6 and you & me`.
17. Multiple choice: which element is a void element? - a) `<p>` - b) `<br>` - c) `<div>` - d) `<span>`
18. Multiple choice: which element marks up a keyboard input? - a) `<code>` - b) `<kbd>` - c) `<samp>` - d) `<var>`
19. Short code question: give a `<nav>` the accessible name "Footer".
20. Multiple choice: which attribute defers loading an off-screen image? - a) `defer` - b) `loading="lazy"` - c) `async` - d) `decoding="lazy"`
21. True or false: one `<h1>` per page is the recommended practice.
22. Multiple choice: which element groups related form controls with a name? - a) `<legend>` - b) `<fieldset>` - c) `<label>` - d) `<optgroup>`
23. Short code question: write a figure with an image and a caption.
24. Multiple choice: which meta tag makes a page responsive? - a) `<meta charset>` - b) `<meta name="viewport" content="width=device-width, initial-scale=1">` - c) `<meta name="description">` - d) `<meta name="robots">`
25. Multiple choice: what does `aria-current="page"` do? - a) reloads the page - b) marks the current item in a navigation - c) sets the page language - d) hides the link

14.1 Final test answers

1. b - the doctype must be the first thing in the document to trigger standards mode.
2. False - `strong` conveys importance, `b` is presentational.
3. b - `alt` is required on `img` (it may be empty for decorative images).
4. `html <a href="https://example.com" target="_blank" rel="noopener">Example site</a>`
5. b - `<li>`.
6. b - `scope="col"`.
7. False - layout belongs to CSS.
8. `<ol type="I" start="5">`
9. b - exactly one.
10. b - `<caption>`.

11. False - a placeholder is a hint, not an accessible name.
12. `html <label for="email">Email</label> <input type="email" id="email" name="email" required>`
13. b - `&`.
14. c - 4.5:1 for normal text.
15. True - `rel="noopener"` (or `noreferrer`) prevents the opened page from touching the original window.
16. `5 < 6 and you & me`
17. b - `br` is void.
18. b - `kbd`.
19. `<nav aria-label="Footer">`
20. b - `loading="lazy"`.
21. True - it defines the page topic in the outline.
22. b - `fieldset` (named by `legend`).
23. `html <figure> <img src="photo.jpg" alt="..."> <figcaption>The caption</figcaption> </figure>`
24. b - the viewport meta tag.
25. b - it announces the current page location to assistive technology.

Sources

Every URL below was consulted while planning and writing this volume. No text, images or code blocks were copied from these pages; all explanations, examples, exercises, tests and answers were written from scratch.

- https://html.com/
- https://html.com/document/
- https://html.com/anchors-links/
- https://html.com/lists/
- https://html.com/images/
- https://html.com/media/
- https://html.com/tables/
- https://html.com/html5/
- https://html.com/semantic-markup/
- https://html.com/forms/
- https://html.com/character-codes/
- https://html.com/tags/
- https://html.com/attributes/
- https://html.com/cheat-sheet/

- https://html.spec.whatwg.org/ - the living HTML standard, used only to check facts such as void elements, content models and the outline algorithm
- https://validator.w3.org/ - used to validate every example in this volume